

An SSRF Journey from "Ignored" to "Triaged"

By J0o1ey · Translated from the Chinese original

0x01 How It Started

Chatting with a friend a couple of days ago, he mentioned a high-paying program on HackerOne where he had found bugs.

I was green with envy.



I checked the vendor's page on HackerOne — the bounties were indeed mouth-watering.

And the bug bounty program launched in April 2015...

An 8-year veteran vendor on HackerOne: few business functions to begin with, picked over for eight years by brilliant hackers from all over the world. The difficulty needs no imagination.

Bug Bounty Program
Launched on Apr 2015

Includes retesting ⓘ
Bounty splitting enabled ⓘ

★ Bookmarked 🔔 Subscribed

Policy Scope **News** Hacktivity Thanks Updates (4) Collaborators

Low	Medium	High	Critical
\$250 - \$4,000	\$4,000 - \$10,000	\$10,000 - \$20,000	\$20,000 - \$35,000

Last updated on February 23, 2022. [View changes](#)

Response Efficiency

13 hrs
Average time to first response

about 1 day
Average time to triage

8 days

But charging at a hard target for the bounty — that is exactly the style a true bug hunter should have.

0x03 Walking the Business Points in Despair, to Finding a Sensitive Request

I collected the business scope from the HackerOne program page and started testing.

I tested the usual web bugs — IDOR and friends — and found zero openings. The filtering and authorization were rock solid, and user identity passed via UUID-type parameters.

First, there was no IDOR; second, even if there were, it would be extremely hard to exploit.

Request

Pretty Raw Hex U2C Chinese

```
1 GET /v1/members/f0b101d8-738b-4b24-b6f6-e6fa2e62702f/notifications?
2 category=ADS&limit=50 HTTP/1.1
3
```

Response

Pretty Raw Hex Render JSON Decoder U2C Chinese

```
1 HTTP/1.1 200 OK
2 date: Fri, 10 Mar 2023 02:27:21 GMT
3 x-request-id: dfa9062e-8ca3-42d5-b8b3-6e9c74905f84
4 content-type: application/json
5 access-control-allow-origin: https://ads.snapchat.com
6 access-control-allow-credentials: true
7 access-control-expose-headers: X-Request-Id
8 x-tr-st-1: 76
9 x-envoy-upstream-service-time: 78
10 vary: Accept-Encoding
11 server: API Gateway
12 strict-transport-security: max-age=31536000; includeSubDomains
13 x-envoy-decorator-operation: /v1/
14 Via: 1.1 google
15 Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000,h3-Q050=":443";
16 ma=2592000,h3-Q046=":443"; ma=2592000,h3-Q043=":443";
17 ma=2592000,quic=":443"; ma=2592000; v="46,43"
18 Connection: close
19 Content-Length: 1110
20 {
21   "request_status": "SUCCESS",
22   "request_id": "dfa9062e-8ca3-42d5-b8b3-6e9c74905f84",
23   "notifications": [
24     {
25       "sub_request_status": "SUCCESS",
26       "notification": {
27         "id": "fcf30a1f-20b6-38fd-be4f-7c82f3ba4b04",
28         "member_id": "f0b101d8-738b-4b24-b6f6-e6fa2e62702f",
29       }
30     }
31   ]
32 }
```

I tested from 9 p.m. to 2 a.m. with nothing to show for it. Every hope extinguished, I was about to wash up and sleep.

But I thought: let me take one last look at Burp's HTTP history — maybe there's a sensitive request I overlooked.

And... there it was: a sensitive GraphQL query request.

```
POST
/agw/graphql?op=UrlReachableVerifierQuery&client_trace_id=09bee58d-8358-4f00-acc0-8d26d0018d32
,rst:1678201703792 HTTP/1.1
Host: xxxxx
Cookie: xxxx
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/110.0
Accept: */*
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate
Content-Type: application/json
Authorization: xxxxx
Content-Length: 386
Origin: https://xxxxx
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-site
Te: trailers
Connection: close
{"operationName":"UrlReachableVerifierQuery","variables":{"url":"http://xxxx.com/"},"query":"query UrlReachableVerifierQuery($url: String!) {\n  verifyUrlReachable(url: $url) {\n    ... on UrlReachableResult {\n      url\n      __typename\n    }\n    ... on GenericError {\n      errorCode\n      message\n      __typename\n    }\n  }\n}"}
```

A url parameter passed in JSON — what to test for, I trust everyone understands.

Immediately fired up Burp Collaborator to test dnslog.

Sure enough, HTTP requests came back from two IPs — but the response echo was pitifully thin:

only a JSON with the keys url and __typename came back.

#	Time	Type	Payload	Source IP address
2	2023-3月-07 15:44:17.372 UTC	HTTP	7ckcm5hx37ks7xsuafkwsshtkflf97xw	34.
4	2023-3月-07 15:47:59.456 UTC	HTTP	7ckcm5hx37ks7xsuafkwsshtkflf97xw	104.
6	2023-3月-07 15:48:09.931 UTC	HTTP	7ckcm5hx37ks7xsuafkwsshtkflf97xw	34.
7	2023-3月-07 15:48:17.896 UTC	HTTP	7ckcm5hx37ks7xsuafkwsshtkflf97xw	34.
9	2023-3月-07 15:49:13.657 UTC	HTTP	7ckcm5hx37ks7xsuafkwsshtkflf97xw	104.
11	2023-3月-07 16:21:51.010 UTC	HTTP	7ckcm5hx37ks7xsuafkwsshtkflf97xw	34.
1	2023-3月-07 15:44:17.158 UTC	DNS	7ckcm5hx37ks7xsuafkwsshtkflf97xw	172.
3	2023-3月-07 15:47:59.229 UTC	DNS	7ckcm5hx37ks7xsuafkwsshtkflf97xw	74.
5	2023-3月-07 15:48:09.704 UTC	DNS	7ckcm5hx37ks7xsuafkwsshtkflf97xw	172.
8	2023-3月-07 15:49:13.429 UTC	DNS	7ckcm5hx37ks7xsuafkwsshtkflf97xw	173.
10	2023-3月-07 16:21:50.785 UTC	DNS	7ckcm5hx37ks7xsuafkwsshtkflf97xw	74.

Description	Request to Collaborator	Response from Collaborator
Pretty	Raw	Hex U2C Chinese
1	GET / HTTP/1.1	
2	user-agent: Mozilla/5.0 (iPhone; CPU iPhone OS 12_4 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148	
3	X-purpose: preview	
4	Host: 7ckcm5hx37ks7xsuafkwsshtkflf97xw.oastify.com	
5	Connection: Keep-Alive	
6	Accept-Encoding: gzip, deflate	
7		
8		

I looked them up: both requesting servers were deployed on Google Cloud. That got me excited instantly.



But thinking again, I deflated. Unlike certain domestic clouds, fetching Google Cloud's metadata requires a specific header.

This SSRF had no echo, and we couldn't get a malicious page's JS to send a request with custom headers. What a pain.

Request example:

```
curl "http://metadata.google.internal/computeMetadata/v1/instance/image" \
-H "Metadata-Flavor: Google"
```

以编程方式查询元数据

在虚拟机中，您可以使用 Linux 上的 `curl` 工具或 Windows 上的 `Invoke-RestMethod` 等工具，以编程方式查询默认或自定义元数据值。

查询单个元数据条目

Linux

Windows

1. 连接到您的 Linux 虚拟机。

2. 在您的 Linux 虚拟机中，使用 [curl 工具](#) 进行查询。

例如，如需查询虚拟机的启动映像，请运行以下查询：

```
user@myinst:~$ curl "http://metadata.google.internal/computeMetadata/v1/instance/image"
```

输出类似于以下内容：

```
projects/rhel-cloud/global/images/rhel-8-v20210122
```

After thinking it over and over, I figured it still counted as a blind SSRF — submit it first and see.

0x04 Instantly Ignored (No Bounty) to Mining New Attack Surface via GraphQL

I submitted the blind SSRF, slept on it, and woke up to find the vendor's security team had marked it Ignored.



Mar 8th (8 days ago)

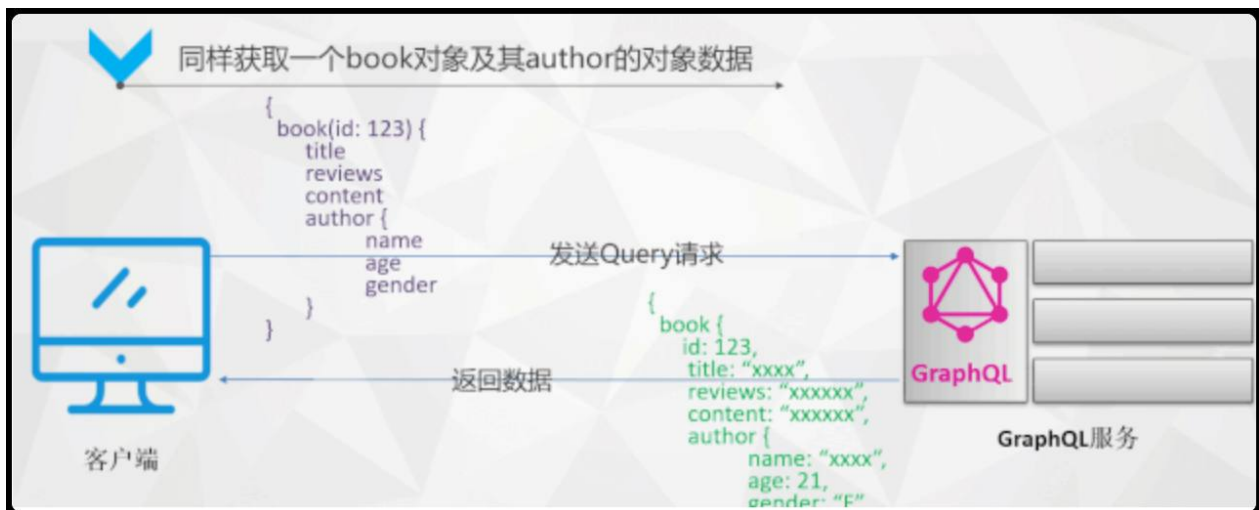
Thank you for your report @j0o1ey,

We have reviewed the information you have provided, and have had similar behavior reported in the past. While this may not be ideal behavior, we do not believe this behavior poses a security risk to our platform.

Although it appears you did receive HTTP and DNS requests at your collaborator server, sending requests to other systems is often a core piece of functionality for many web applications. For example, when posting a link on many services, those services will show a preview of the content that exists at that site. In order to do this, these services need to make a request to the specified site to retrieve the preview information.

Bloody hell. A blind SSRF clearly wasn't going to make money; we had to dig out a new attack surface.

(If you're not familiar with GraphQL, the picture below vividly illustrates the difference between RESTful and GraphQL interfaces: the top shows RESTful-style requests, the bottom GraphQL-style.)



Since this endpoint queries via GraphQL,

we can define custom query columns (params) — if a column exists, the query returns a meaningful result for it. No hesitation: FUZZ.



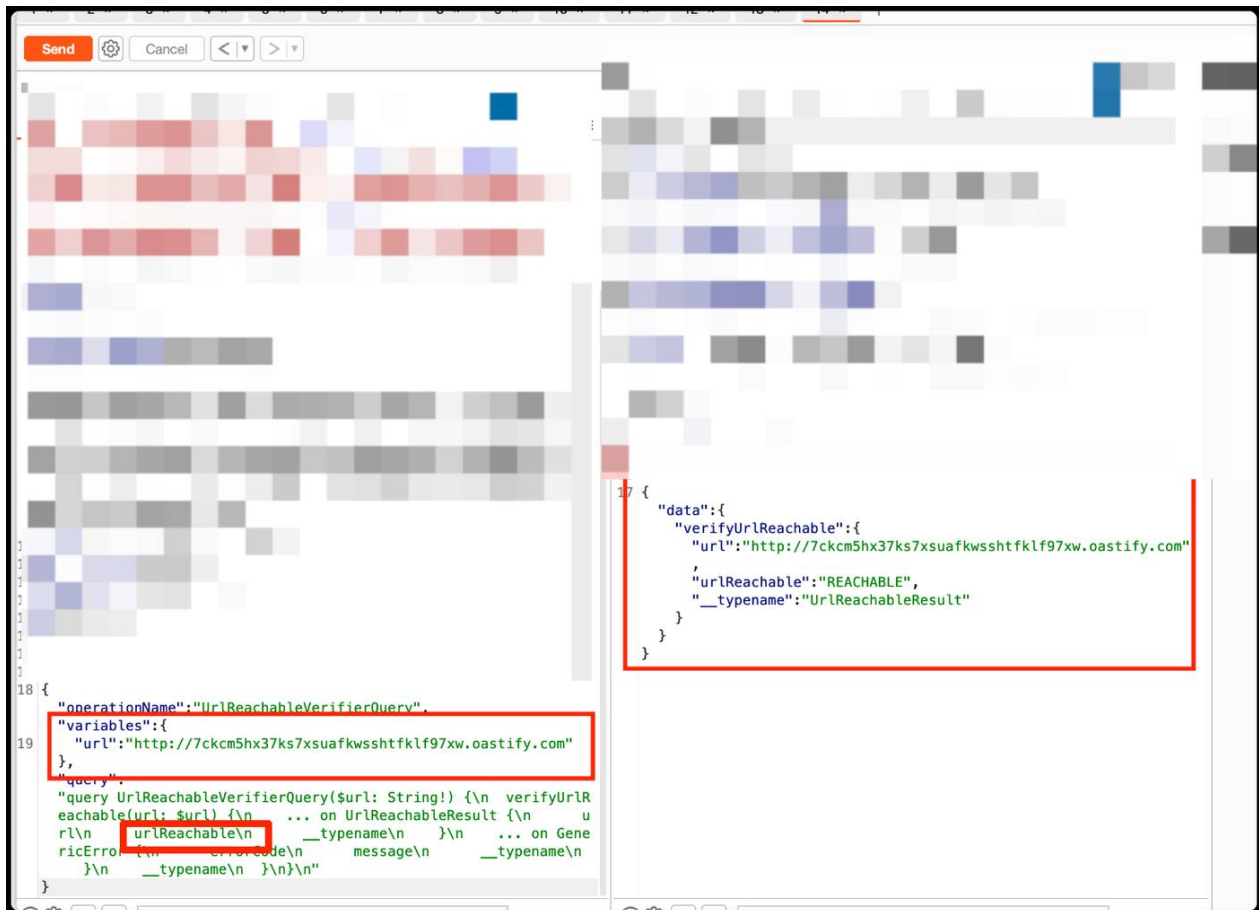
The result was heartbreaking: absolutely jack shit (I forgot to save the screenshot at the time).

Back into despair. But staring closely at the op parameter of the GraphQL request gave me an idea.



The `op` field was `UrlReachableVerifierQuery` — why not try using it as a query column?

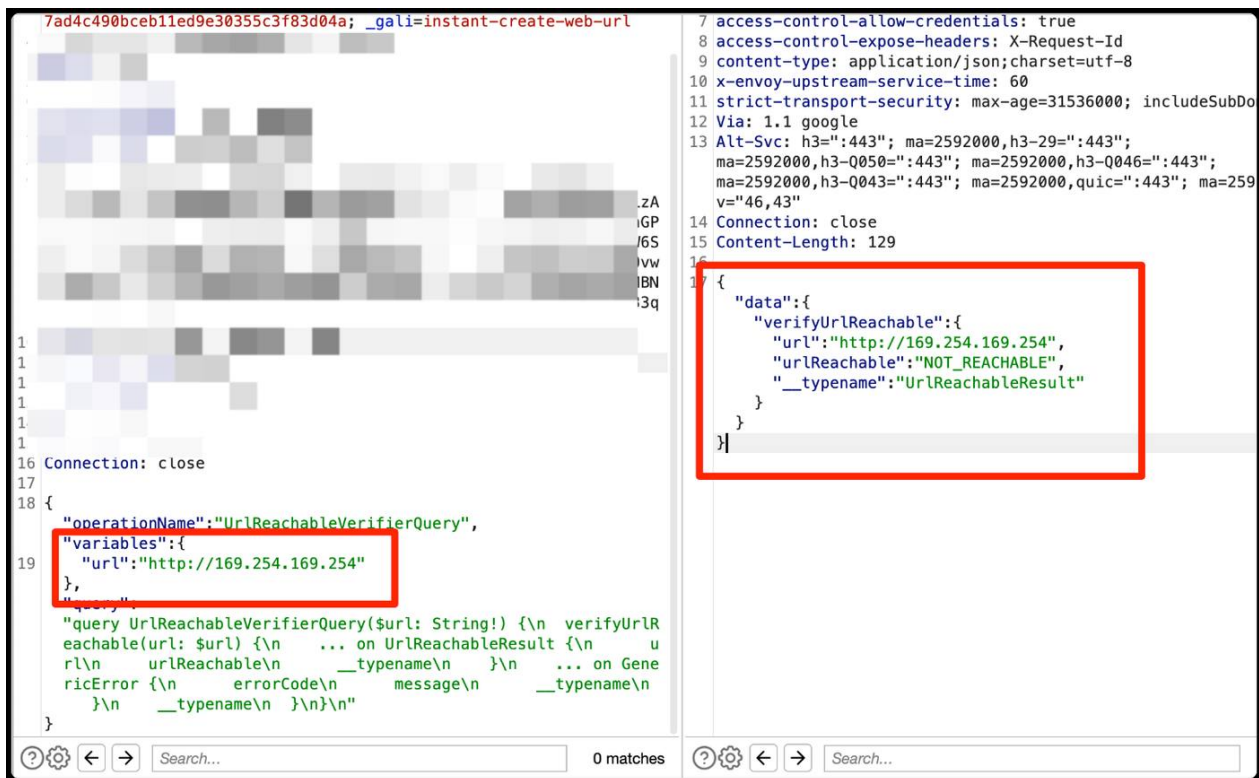
When I tested the field `UrlReachable`, the response suddenly echoed back a meaningful "Reachable".



Nice — now we could use this interface to probe intranet port availability.

I went straight for the Google Cloud metadata address to test connectivity.

To my surprise it flat-out returned "Not_Reachable"...



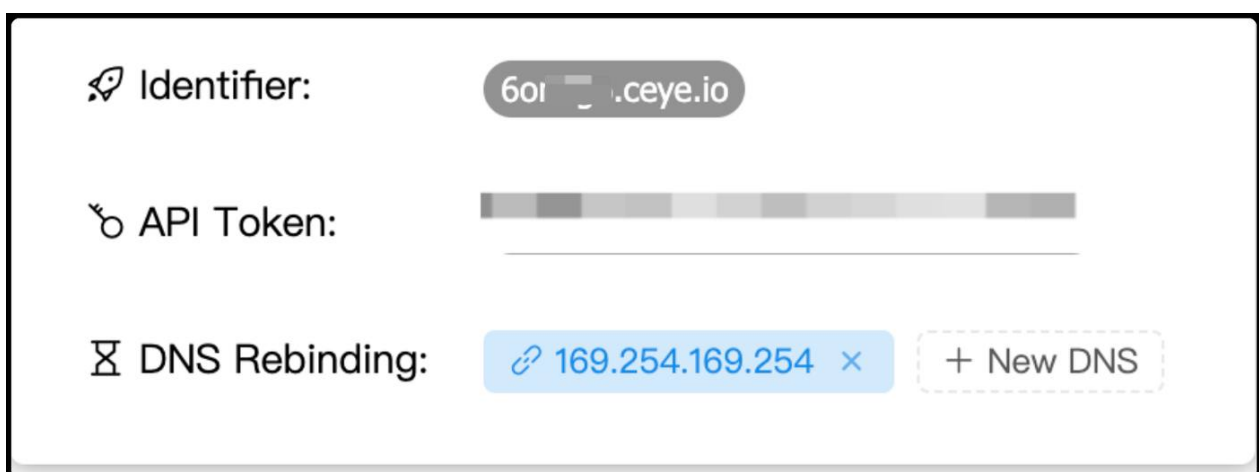
Time for some bypass craftsmanship.

0x05 Bypassing the SSRF Restrictions to Probe the Intranet

Tried 302 redirects, short links, and friends — none worked.

So I brought out DNS rebinding.

Configured the DNS rebinding IP on ceye (the Google Cloud metadata IP is 169.254.169.254 — using it to verify intranet connectivity).

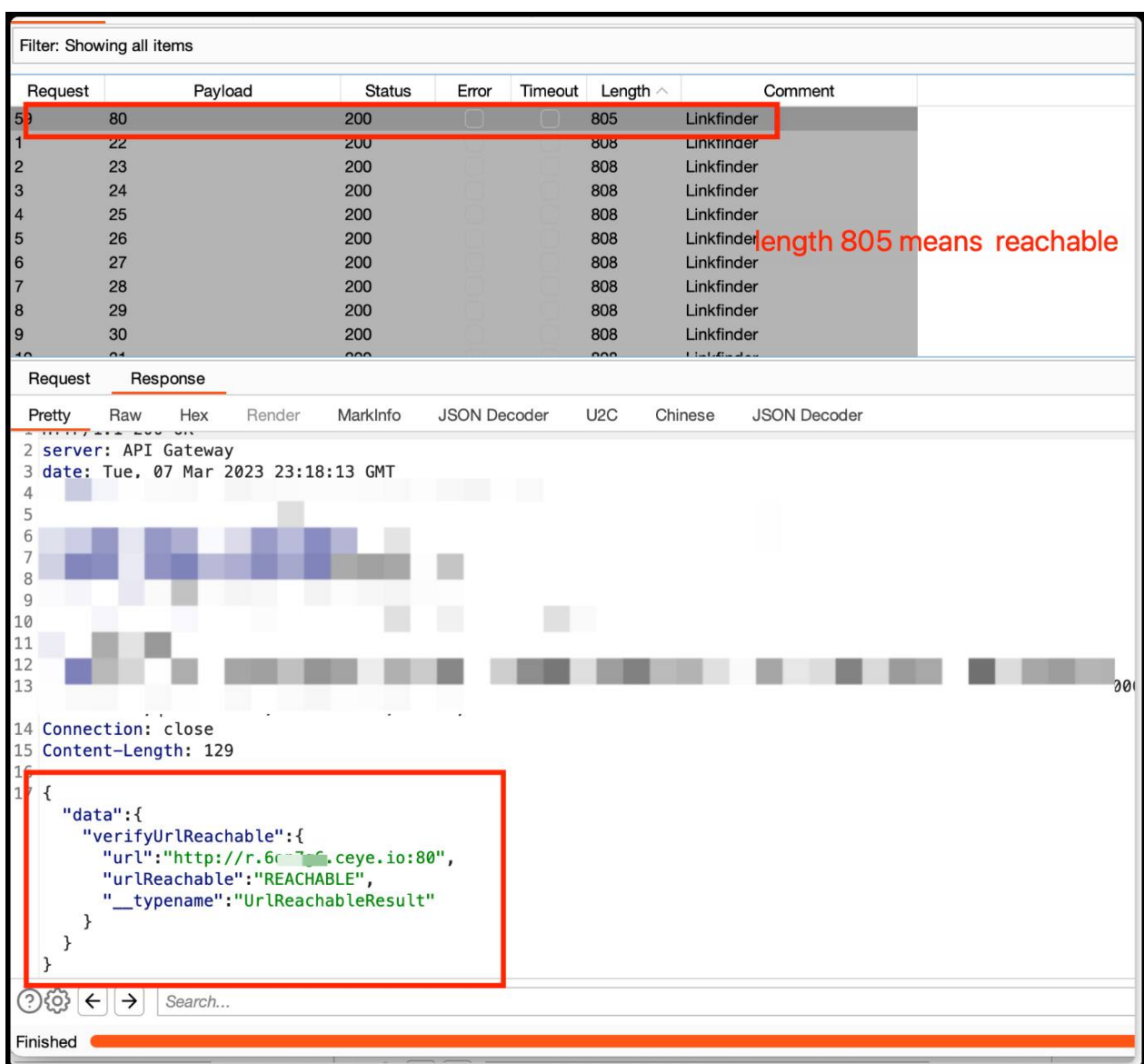


Straight-up DNS rebinding to bypass the SSRF restriction.



And there it was — a "Reachable" result!

Then the routine: probe port connectivity.





Port 80 reachable, other ports Not_Reachable — proof enough that this SSRF could probe the intranet.

0x06 Resubmitting, to Triaged

After resubmitting, the vulnerability was triaged.

